

Non-Mutating Quicksort

```
def quicksort(lst: list) -> list:
    if len(lst) < 2:
        return lst[:]
    else:
        pivot = lst[0]
        smaller, bigger = _partition(lst[1:], pivot)
        smaller_sorted = quicksort(smaller)
        bigger_sorted = quicksort(bigger)
        return smaller_sorted + [pivot] + bigger_sorted
```

- Return a new sorted list, but does not mutate the original list

Inplace Quicksort

- The key change is inplace `_partition()`

Simulating Slicing with Indexes

- We often want to operate on just part of a list
 - `f(lst[start:end])`
- Rather than creating a new list object, we can pass in the indexes for efficiency
 - `f(lst, start, end)`
- Connecting to `_in_place_partition()`...
 - `_in_place_partition(lst)` ⇒ `_in_place_partition(lst, start, end)`
 - `quicksort(lst)` ⇒ `_in_place_quicksort(lst, start, end)`